

```
@someAttribute.deleter
def someAttribute(self): # This is the "deleter" method.
    del self._someAttribute

obj = ClassWithProperties()
print(obj.someAttribute) # Prints 'some initial value'
obj.someAttribute = 'changed value'
print(obj.someAttribute) # Prints 'changed value'
del obj.someAttribute # Deletes the _someAttribute attribute.
```

This program's output is the same as the *regularAttributeExample.py* code, because they effectively do the same task: they print an object's initial attribute and then update that attribute and print it again.

But notice that the code outside the class never directly accesses the `_someAttribute` attribute (it's private, after all). Instead, the outside code accesses the `someAttribute` property. What this property actually consists of is a bit abstract: the getter, setter, and deleter methods combined make up the property. When we rename an attribute named `someAttribute` to `_someAttribute` while creating getter, setter, and deleter methods for it, we call this the `someAttribute` property.

In this context, the `_someAttribute` attribute is called a *backing field* or *backing variable* and is the attribute on which the property is based. Most, but not all, properties use a backing variable. We'll create a property without a backing variable in "Read-Only Properties" later in this chapter.

You never call the getter, setter, and deleter methods in your code because Python does it for you under the following circumstances:

- When Python runs code that accesses a property, such as `print(obj.someAttribute)`, behind the scenes, it calls the getter method and uses the returned value.
- When Python runs an assignment statement with a property, such as `obj.someAttribute = 'changed value'`, behind the scenes, it calls the setter method, passing the 'changed value' string for the `value` parameter.
- When Python runs a `del` statement with a property, such as `del obj.someAttribute`, behind the scenes, it calls the deleter method.